

Zomato / Swiggy - Food Delivery System Design

YouTube-Style Interview Guide

Print Settings: Landscape mode, monospace font (Courier New 9-10pt), narrow margins

TABLE OF CONTENTS

1. [Requirements Gathering](#)
 2. [Capacity Estimation](#)
 3. [High-Level Architecture](#)
 4. [Core Workflows](#)
 5. [Scalability & Optimizations](#)
 6. [Interview Q&A](#)
-

SECTION 1: REQUIREMENTS GATHERING

1.1 Functional Requirements

- ✅ Core Features:
 1. Customer: Browse restaurants, place orders, track delivery
 2. Restaurant: Manage menus, accept/reject orders, update order status
 3. Delivery Agent: Accept deliveries, update location, mark delivered
 4. Payment: Multiple payment methods, split settlements
 5. Search: Find restaurants by location, cuisine, rating
 6. Notifications: Real-time updates to all parties
 7. Rating & Review: Rate food quality and delivery
- ❌ Out of Scope:
 - Scheduled orders
 - Group ordering
 - Subscription models
 - Loyalty programs

1.2 Non-Functional Requirements

Availability:	99.99% uptime
Scalability:	Handle millions of concurrent users
Latency:	Search < 500ms, Order placement < 2s, Location updates < 3s
Consistency:	Strong for orders/payments, Eventual for search
Real-time:	Live tracking with < 5s update intervals

SECTION 2: CAPACITY ESTIMATION

Assumptions:

- 50 million DAU (Daily Active Users)
- Average 2 orders per user per week
- Peak hour traffic: 20% of daily orders

Calculations:

Orders per day: 14M orders

Orders per second:

- Average: 162 orders/sec
- Peak: 810 orders/sec

Read:Write ratio: 100:1 (browsing vs ordering)

Search queries: 16,200 QPS

Storage:

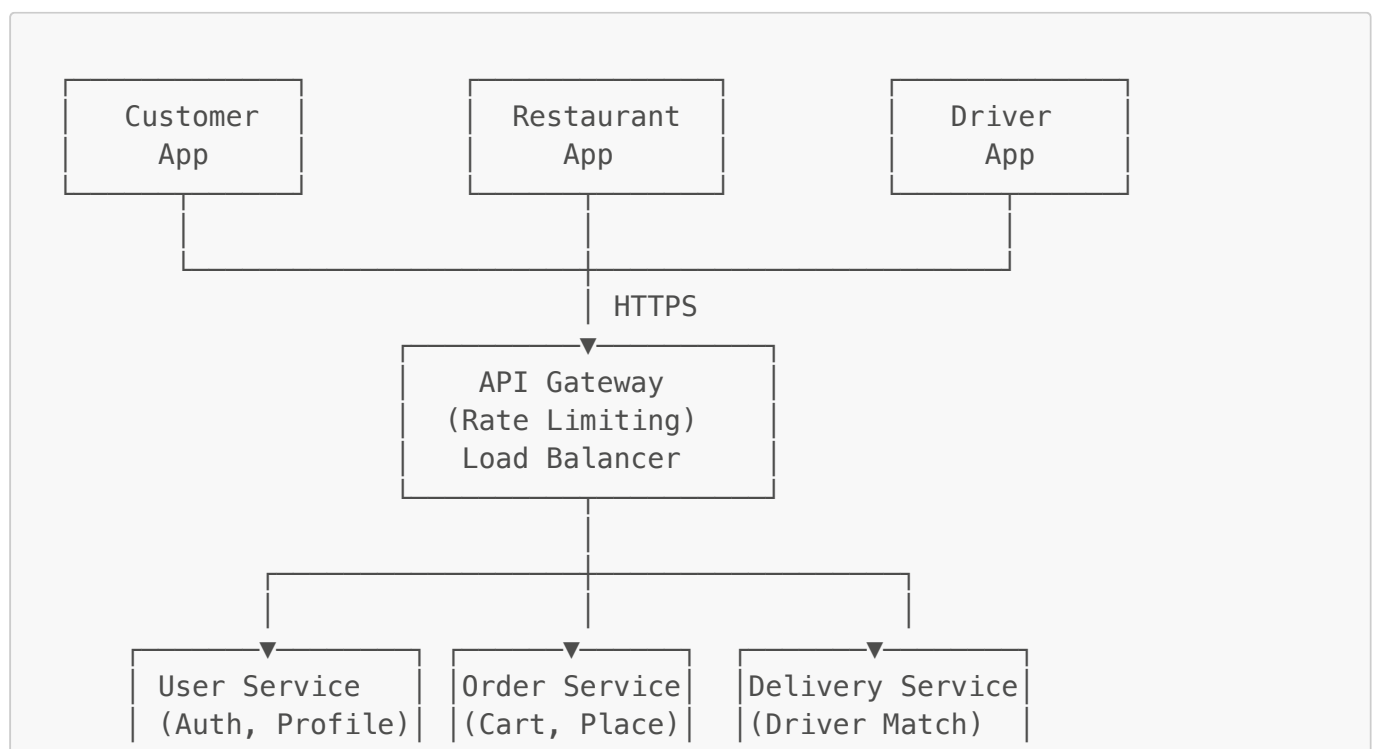
- Per order: 8 KB (metadata + items + tracking)
- Daily: $14M \times 8KB = 112$ GB/day
- Annual: 40 TB/year
- With replication (3x): 120 TB/year
- 5-year total: ~600 TB

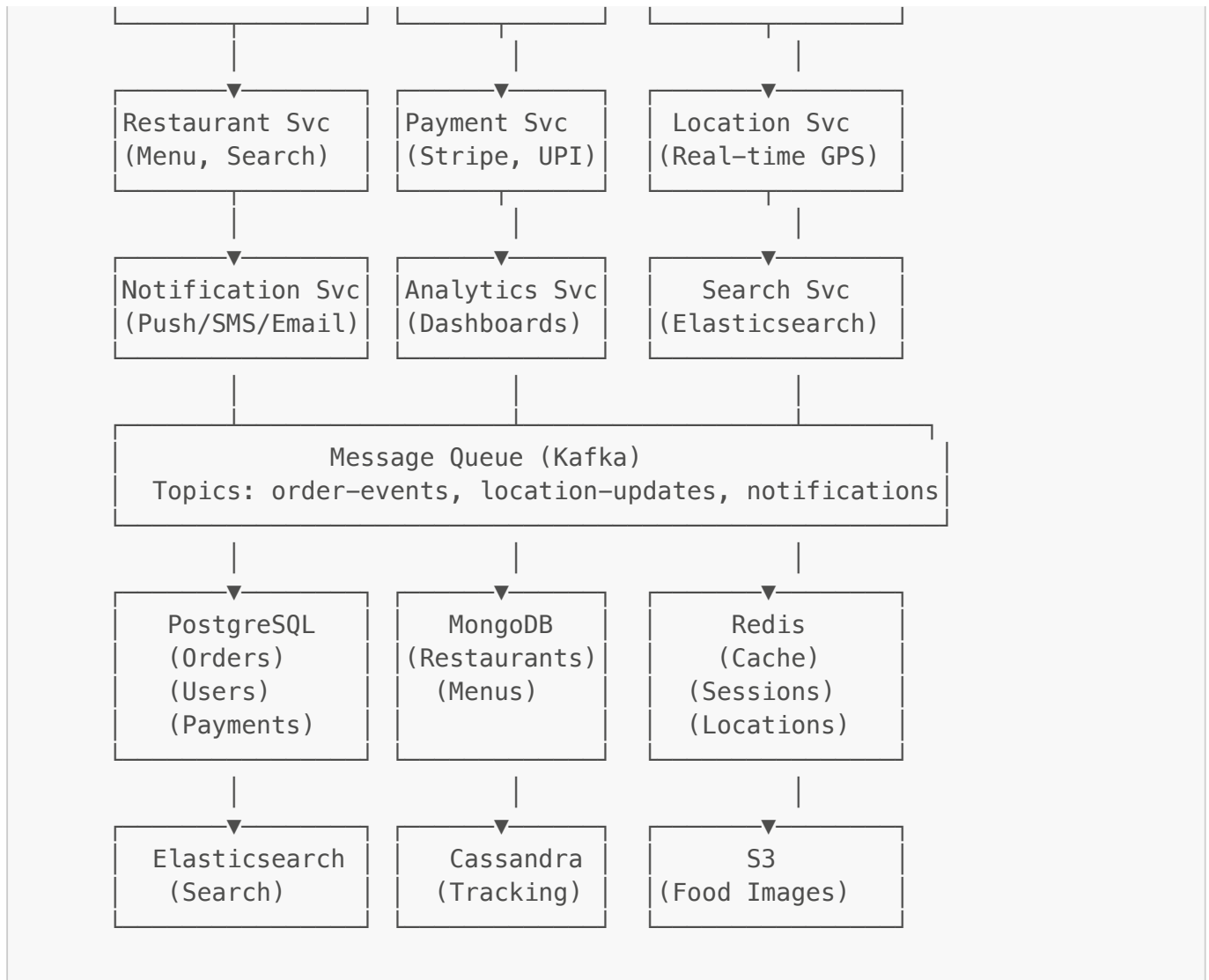
Bandwidth:

- Incoming: 15 MB/s (orders + location updates)
- Outgoing: 3 GB/s (search results + tracking)

SECTION 3: HIGH-LEVEL ARCHITECTURE

3.1 System Architecture Diagram





3.2 Microservices Breakdown

User Service (Node.js/Go):

- Authentication & authorization (JWT tokens)
- Profile management (customer, restaurant, driver)
- Address management with geocoding
- Database: PostgreSQL + Redis (sessions)

Restaurant Service (Java/Spring Boot):

- Restaurant CRUD operations
- Menu management with item variants
- Availability and timing management
- Database: MongoDB (flexible menu schema)
- Search: Elasticsearch

Order Service (Java/Spring Boot):

- Shopping cart management
- Order placement and validation
- Order state machine management

- Order history and tracking
- Database: PostgreSQL (ACID transactions)
- Cache: Redis for active orders

Delivery Service (Python/Go):

- Driver onboarding and verification
- Driver availability management
- Order-driver matching algorithm
- Route optimization
- Database: PostgreSQL + Redis (geospatial)

Location Service (Go):

- Real-time driver location tracking
- Customer order tracking
- ETA calculation
- Database: Cassandra (time-series data)
- Real-time: WebSockets

Payment Service (Java/Spring Boot):

- Payment processing (Stripe, PayPal, UPI)
- Transaction recording
- Split payments (platform fee, restaurant revenue)
- Driver payouts and refunds
- Database: PostgreSQL (PCI-DSS compliance)

Search Service (Python):

- Restaurant and menu item search
- Filters: cuisine, price, rating, delivery time
- Autocomplete suggestions
- Personalized ranking with ML
- Search Engine: Elasticsearch

Notification Service (Node.js):

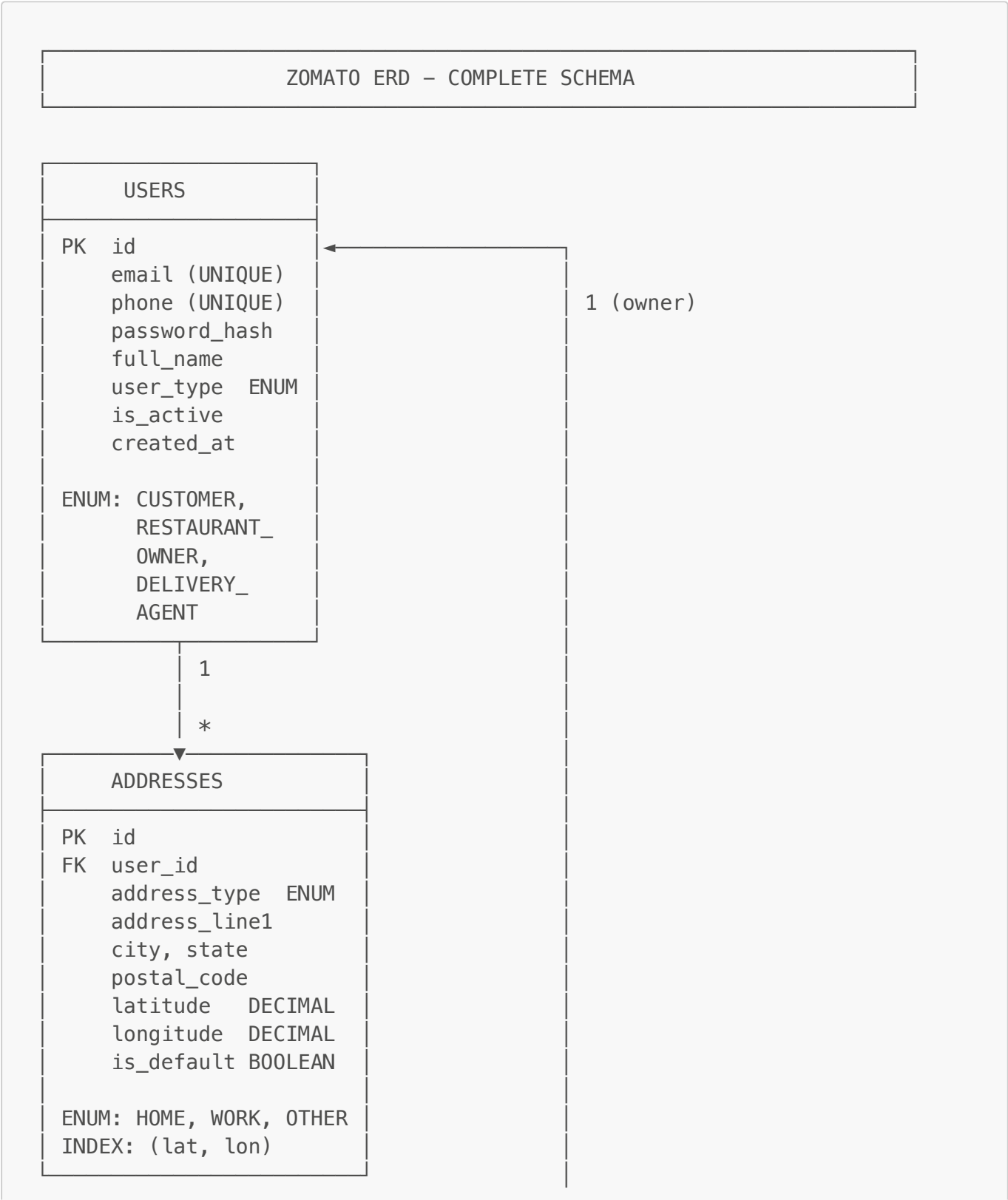
- Push notifications (FCM, APNs)
- SMS (Twilio)
- Email (SendGrid)
- Queue: Kafka

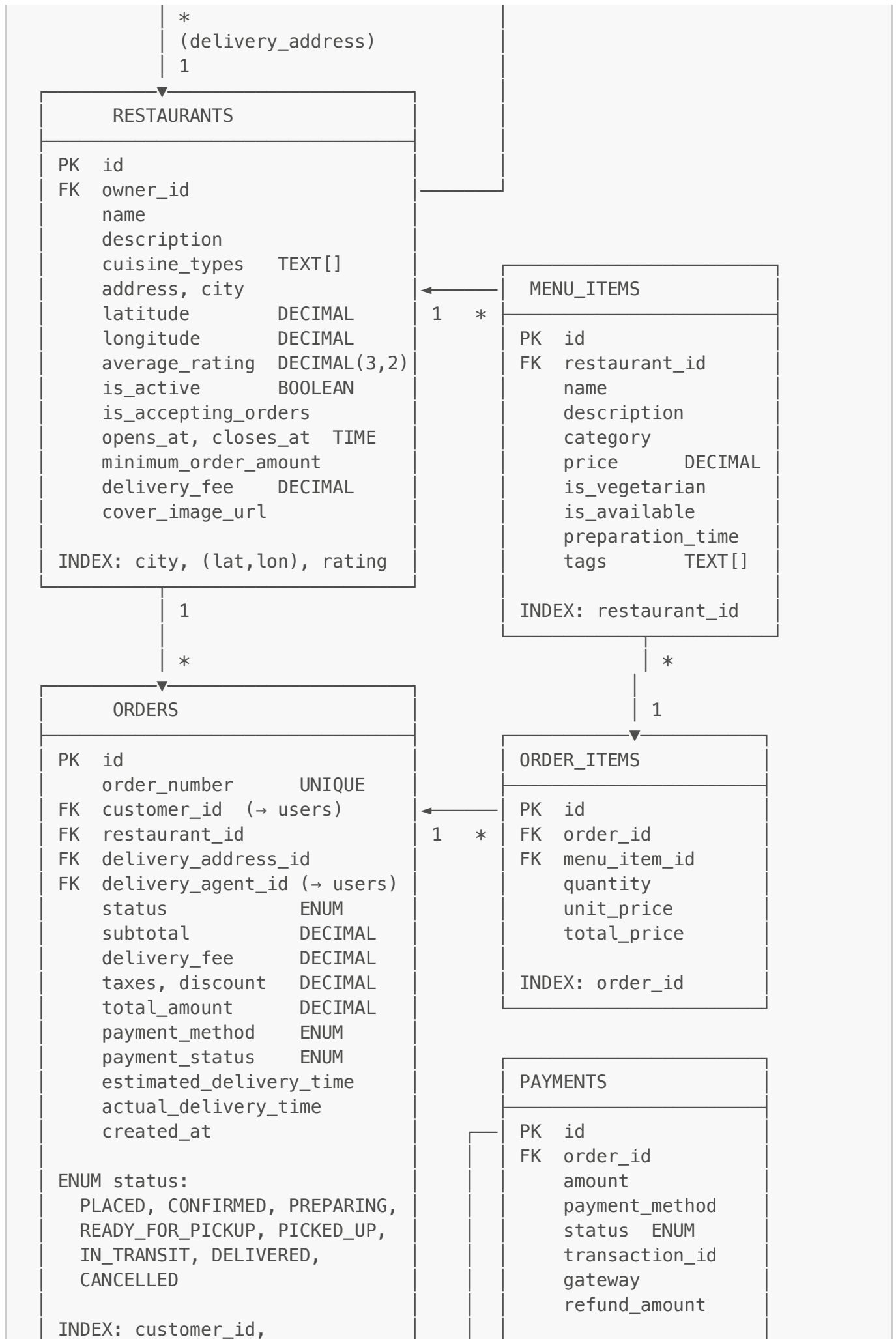
3.3 Database Strategy

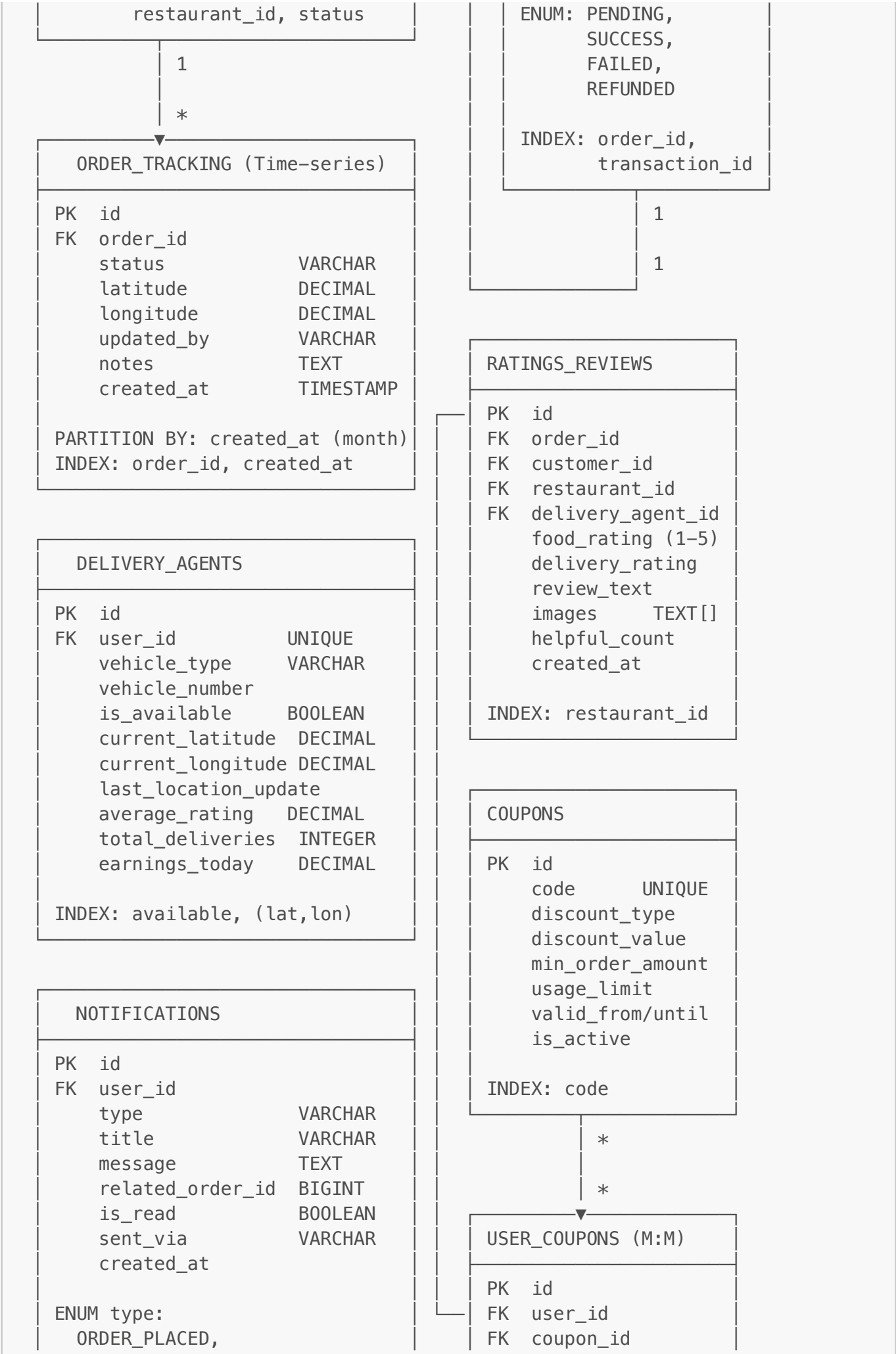
Database	Type	Used For	Why
PostgreSQL	Relational	Orders, Users, Payments	ACID compliance for transactions
Redis	Cache	Menus, Sessions, Locations	High-frequency reads (100k+ QPS)

Database	Type	Used For	Why
MongoDB	Document	Restaurants, Menus	Flexible schema per restaurant type
Elasticsearch	Search	Restaurant search	Full-text + geo-distance queries
Cassandra	Time-series	Location tracking	Append-only audit logs
S3	Object Storage	Food images	Scalable image storage

3.4 Complete Entity Relationship Diagram







ORDER_CONFIRMED,
OUT_FOR_DELIVERY,
DELIVERED, CANCELLED

INDEX: user_id, is_read

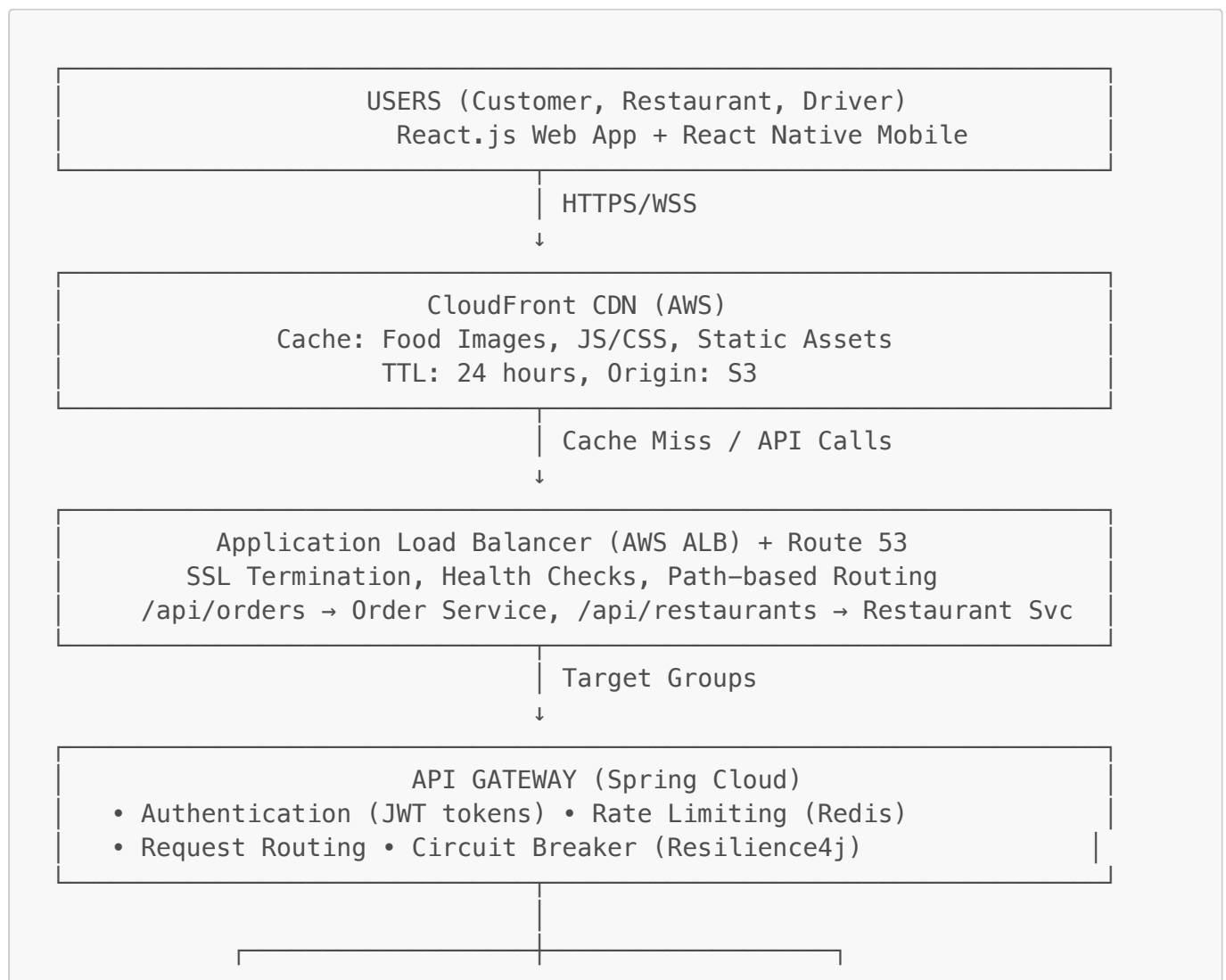
FK order_id
used_at

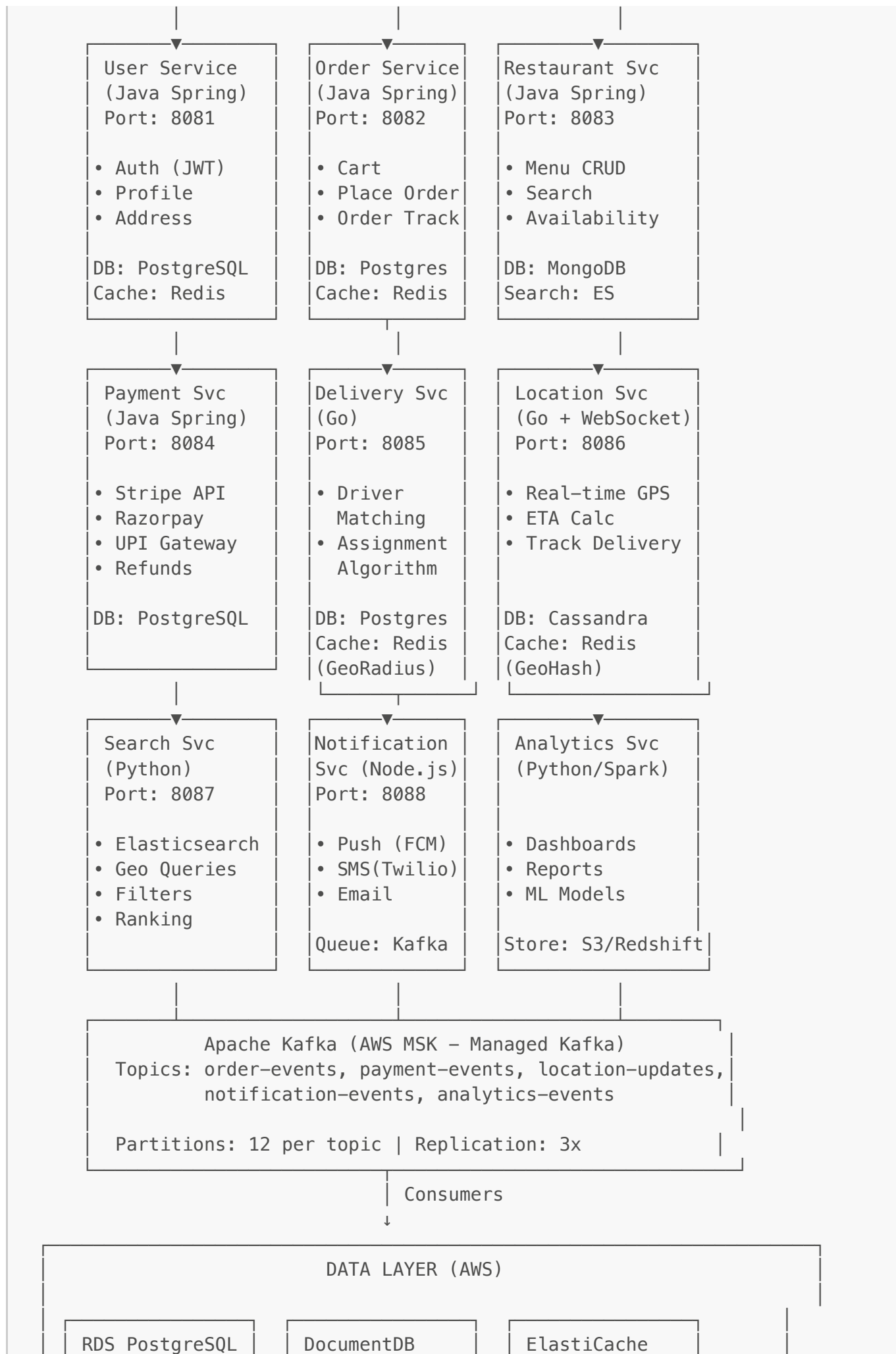
UNIQUE: (user_id,
coupon_id,
order_id)

CARDINALITY SUMMARY:

- users (1) → (*) addresses
- users (1) → (*) restaurants (as owner)
- restaurants (1) → (*) menu_items
- users (1) → (*) orders (as customer)
- orders (*) → (1) restaurants
- orders (*) → (1) delivery_agents
- orders (1) → (*) order_items
- orders (1) → (1) payments
- orders (1) → (*) order_tracking
- orders (1) → (*) ratings_reviews
- users (*) ↔ (*) coupons (via user_coupons)

3.5 Complete Tech Architecture (AWS + Java + React)





(AWS) Master + 2 Read Replicas Tables: - users - orders - payments - ratings Multi-AZ Auto-backup	(MongoDB API) Sharded by restaurant_id Collections: - restaurants - menus - details Multi-AZ Auto-backup	(Redis) 6-node cluster (3 primary + 3 replica) Keys: • active_orders • sessions • search_cache • drivers: available (GeoSpatial)
Elasticsearch (AWS OpenSrcH) 3-node cluster Indexes: - restaurants - menu_items Geo queries Full-text	Keyspaces (Cassandra API) Time-series location data Tables: - location_log TTL: 7 days Partition by driver_id	S3 (Object Store) Buckets: • food-images • backups • logs Lifecycle: Std → IA → Glac CDN Origin

INFRASTRUCTURE & DEPLOYMENT

- Container Orchestration: AWS ECS/EKS (Kubernetes)
- Service Discovery: AWS Cloud Map
- Secrets Management: AWS Secrets Manager
- Monitoring: CloudWatch + Prometheus + Grafana
- Logging: ELK Stack (Elasticsearch, Logstash, Kibana)
- Tracing: AWS X-Ray / Jaeger
- CI/CD: AWS CodePipeline, GitHub Actions
- IaC: Terraform / AWS CloudFormation

TECH STACK SUMMARY:

Frontend: React.js (Web), React Native (Mobile)

API Gateway: Spring Cloud Gateway (Java)

Backend: Java/Spring Boot (Order, Payment, Restaurant, User)
Go (Delivery, Location – high throughput)
Python (Search, Analytics, ML)
Node.js (Notification – async I/O)

Databases: PostgreSQL, MongoDB, Redis, Cassandra, Elasticsearch

Message Queue: Apache Kafka (AWS MSK)
Cloud: AWS (ECS/EKS, RDS, S3, CloudFront, ALB, Route53)

3.6 Design Patterns Applied

Architectural Patterns:

- Microservices Architecture (independent scaling)
- Event-Driven Architecture (Kafka-based Saga pattern)
- API Gateway Pattern (single entry point)
- CQRS (write to master, read from replicas)

Application Patterns:

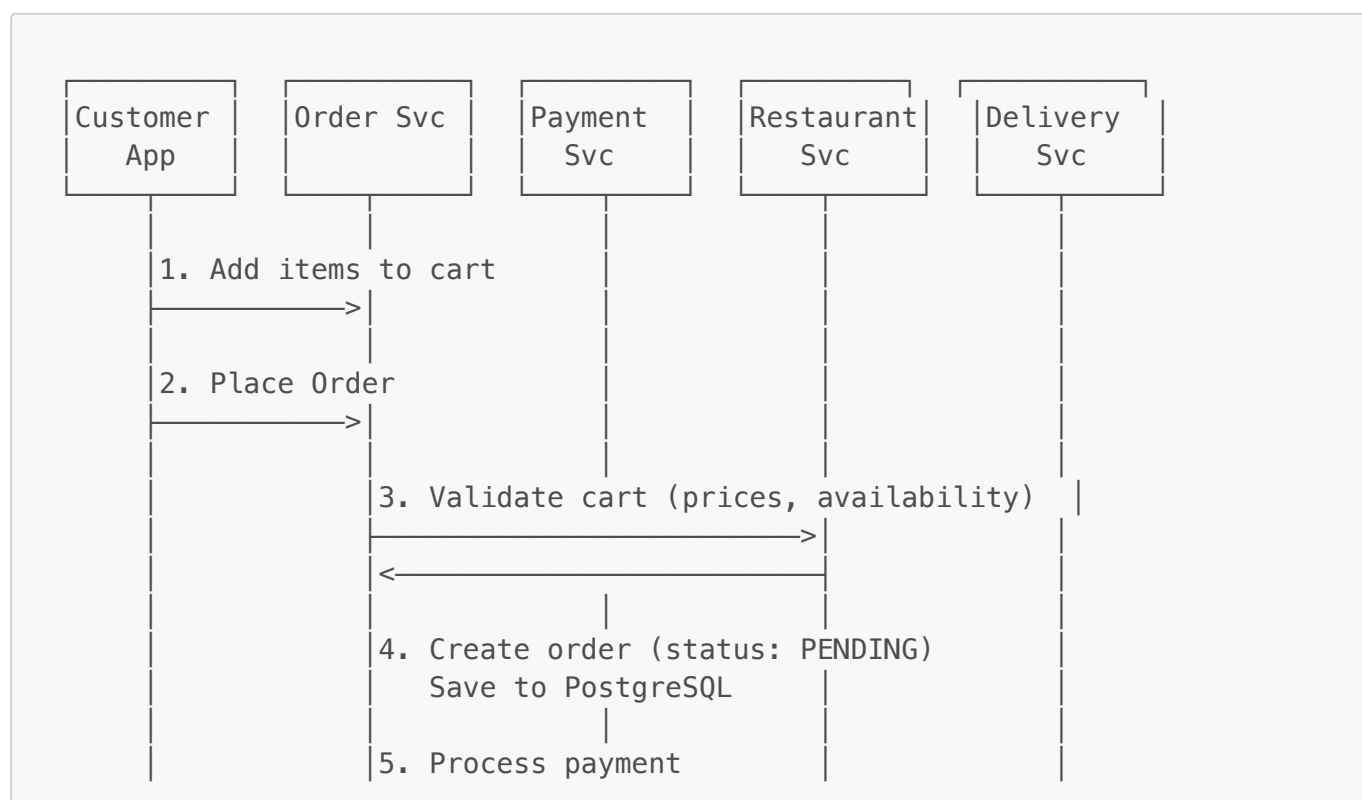
- State Pattern (order state machine)
- Strategy Pattern (driver assignment algorithms)
- Observer Pattern (real-time notifications)
- Circuit Breaker Pattern (payment gateway failures)
- Factory Pattern (payment gateway selection)

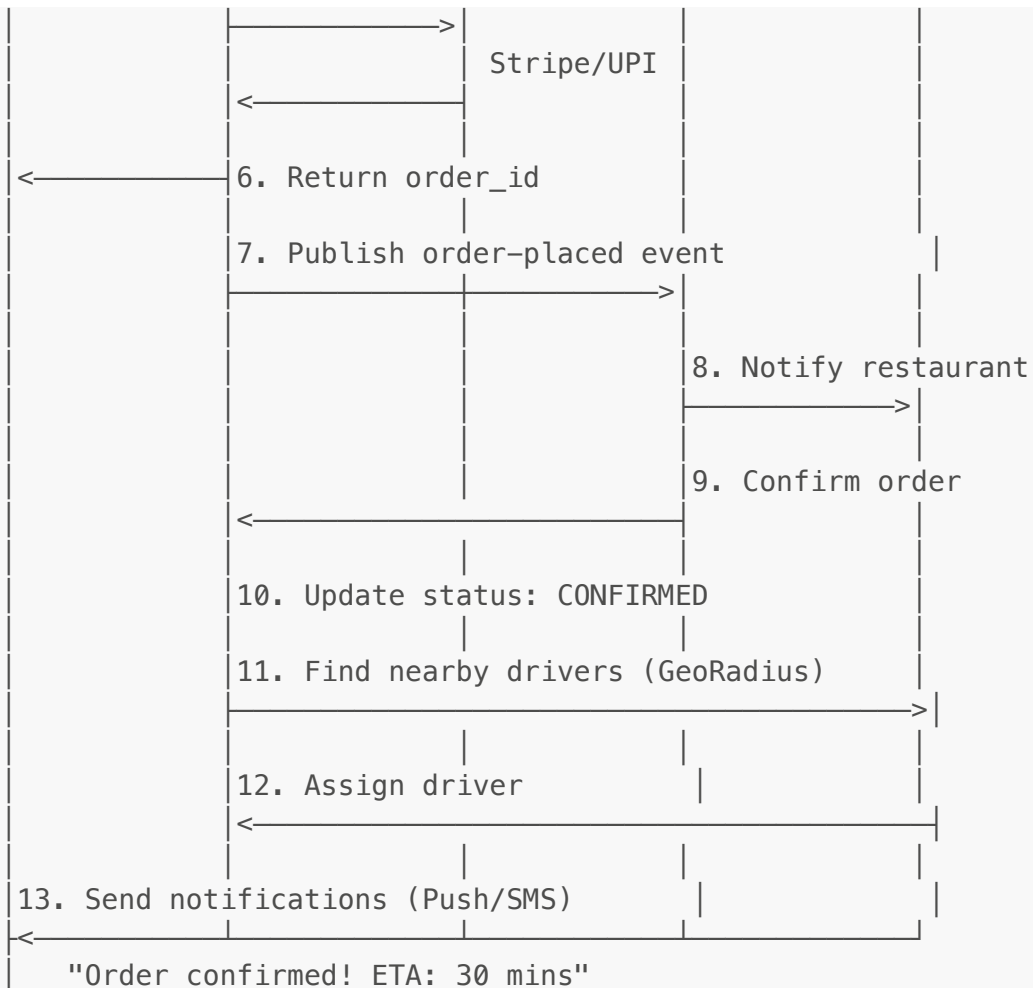
Caching Patterns:

- Cache-Aside (lazy loading)
- Write-Through (order updates)
- Refresh-Ahead (popular restaurants)

SECTION 4: CORE WORKFLOWS

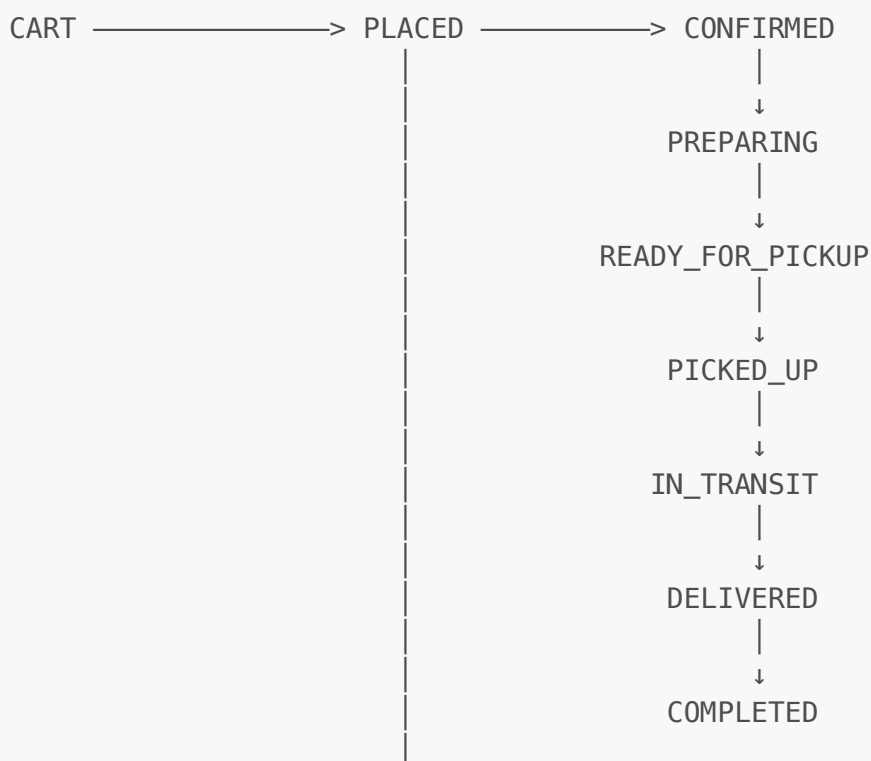
4.1 Order Placement Flow





4.2 Order State Machine

Order Status Flow:



↓
CANCELLED

Cancellation Rules:

- PLACED → CONFIRMED: Full refund
- CONFIRMED → PREPARING: Full refund
- PREPARING → READY: Partial refund (restaurant charges)
- PICKED_UP+: No refund

4.3 Driver Assignment Algorithm

Step 1: Find nearby drivers (Redis GeoRadius)

```
# Find drivers within 5km radius
GEORADIUS drivers:available {restaurant_lat} {restaurant_lon} 5 km WITHDIST
```

Step 2: Score drivers

```
score = (
    distance_weight * (1 - normalized_distance) + # 40% - Closer is better
    rating_weight * normalized_rating +           # 20% - Higher rating
    better
    utilization_weight * (1 - utilization) +      # 20% - Less busy
    better
    eta_weight * (1 - normalized_eta)             # 20% - Faster ETA
    better
)
```

Step 3: Broadcast to top 5 drivers (Kafka)

```
Topic: driver-assignment-requests
Message: {order_id, restaurant, customer_address, earning, timeout: 30s}

First driver to accept gets the order.
If timeout → re-broadcast to next batch.
```

4.4 Real-Time Location Tracking

Driver Side (Mobile App):

```
Every 10 seconds:
1. Get GPS coordinates (lat, lon, accuracy)
2. Batch with last 3 updates
```

3. Send to Location Service via HTTP/gRPC
4. Service updates Redis + Cassandra

Customer Side (WebSocket):

1. Customer connects: `ws://api/track/{order_id}`
2. Server fetches `driver_id` from order
3. Subscribe to driver location updates (Redis Pub/Sub)
4. Push location to customer every 5 seconds
5. Calculate ETA using Google Maps Distance Matrix API
6. Send update: `{lat, lon, eta_minutes, distance_remaining}`

4.5 Restaurant Search Flow

Elasticsearch Query:**Query Structure:**

1. Geo Filter: Within 5km delivery radius
2. Filters: `cuisine`, `rating > 4.0`, `is_open = true`
3. Scoring:
 - Text relevance (BM25): 30%
 - Rating: 25%
 - Delivery time: 20%
 - Popularity: 15%
 - Distance: 10%
4. Personalization: Boost based on order history

Cache: Redis for popular queries (TTL: 5 min)

SECTION 5: SCALABILITY & OPTIMIZATIONS

5.1 Horizontal Scaling

Service Layer:

- Stateless microservices behind load balancers
- Auto-scaling based on CPU/memory (Kubernetes HPA)
- Pod count: min 3, max 50

Database Layer:

- PostgreSQL: Primary-Replica replication (1 primary + 3 read replicas)
- MongoDB: Sharding by `restaurant_id` (4 shards)
- Cassandra: Partition by `driver_id`
- Redis Cluster: 6 nodes (3 primary + 3 replica)

5.2 Caching Strategy

Multi-level Caching:

1. **CDN:** Food images, static assets (CloudFront)
2. **Redis:** Hot data (90% cache hit rate)
 - Restaurant menus: TTL 1 hour
 - Active orders: TTL 2 hours
 - Search results: TTL 5 minutes
3. **Application Cache:** In-memory (Caffeine)

Cache Invalidation:

- Write-through for critical data (order status)
- TTL-based for non-critical (menus)
- Event-driven invalidation (Kafka)

Redis Key Patterns:

- active_orders:{order_id}	→ Order JSON (TTL: 2 hours)
- restaurant:{id}	→ Restaurant JSON (TTL: 1 hour)
- driver_location:{driver_id}	→ GeoHash (TTL: 30 sec)
- search_results:{query_hash}	→ Results (TTL: 5 min)
- user_session:{token}	→ User context (TTL: 7 days)

Geospatial Commands:

```
GEOADD drivers:available 77.5946 12.9716 driver_123
GEORADIUS drivers:available 77.5946 12.9716 5 km
```

5.3 Real-Time System at Scale

Location Updates (100K active drivers):

Write Path:

1. Mobile app batches updates (every 10s)
2. Load balancer → Go service (handles 50K writes/sec)
3. Batch write to Cassandra (100 updates per write)
4. Update Redis cache (GEOADD)
5. Publish to Kafka (for analytics)

Read Path:

1. Customer subscribes via WebSocket
2. Fetch driver location from Redis (GEOPOS)
3. Calculate ETA (Google Maps API)
4. Push update every 5s

WebSocket Scaling:

- Sticky sessions at load balancer
- Redis Pub/Sub for message broadcasting
- Horizontal scaling with connection pooling

5.4 Fault Tolerance

Circuit Breaker Pattern:**Payment Service:**

- If 50% requests fail in 10s → Open circuit
- Fallback: Queue payment for later processing
- Try again after 30s

Restaurant Service:

- If unavailable → Serve stale cache
- Mark restaurant as "temporarily unavailable"

Retry with Exponential Backoff:**Notification Service:**

- Failed push notification → Retry after 1s, 2s, 4s, 8s
- Max 3 retries
- After 3 failures → Move to dead letter queue

Data Durability:

- Database replication (sync for orders, async for reads)
- Cross-region backups (daily to S3)
- Point-in-time recovery (PITR)
- Event sourcing for order state

5.5 Consistency Model

Strong Consistency (Cannot compromise):

- Order placement: Database transactions
- Payment: Distributed transaction
- Driver assignment: Pessimistic locking

Eventual Consistency (Acceptable):

- Search results: Slightly stale OK
- Ratings: Update async
- Analytics: Delayed OK

5.6 API Design (Key Endpoints)

Restaurant APIs:

```
GET /api/v1/restaurants/search
Query: lat, lng, cuisine, rating_min, delivery_time_max, page

GET /api/v1/restaurants/{restaurant_id}/menu
```

Order APIs:

```
POST /api/v1/orders
Body: {restaurant_id, items, delivery_address_id, payment_method_id}

GET /api/v1/orders/{order_id}

PUT /api/v1/orders/{order_id}/cancel
```

Tracking APIs:

```
GET /api/v1/orders/{order_id}/track

WebSocket: ws://api/v1/track/{order_id}
Events: location_update, status_change, eta_update
```

Driver APIs:

```
PUT /api/v1/drivers/status
POST /api/v1/drivers/location
PUT /api/v1/drivers/orders/{order_id}/pickup
PUT /api/v1/drivers/orders/{order_id}/deliver
```

SECTION 6: INTERVIEW Q&A

Q1: How do you handle the driver assignment problem efficiently?

Answer: "Driver assignment is a complex optimization problem. Here's my approach:

Algorithm:

1. **Spatial Indexing:** Use Redis GEORADIUS to find drivers within 5km
2. **Scoring System:**
 - Distance: 40% (closer is better)
 - Driver rating: 20%
 - Current utilization: 20% (less busy better)

- Estimated delivery time: 20%

3. Assignment Flow:

- Broadcast to top 5 drivers with 30s timeout
- First to accept gets the order
- If timeout, re-broadcast to next batch

Advanced Optimizations:

- **Batch Assignment:** Assign multiple orders to one driver
- **Predictive Positioning:** ML predicts demand hotspots, pre-position drivers
- **Dynamic Zones:** Adjust zone boundaries based on real-time demand"

Q2: How do you ensure consistency in order lifecycle across services?

Answer: "We use **Saga Pattern** with event-driven choreography:

Flow:

```
Order Placed → Payment (charge)
                → Restaurant (notify)
                → Delivery (assign driver)
                → Notification (send updates)
```

Implementation:

- **Event Sourcing:** Store all state changes as events in Kafka
- **Compensating Transactions:** Rollback on failure
 - Payment failed → Cancel order → Refund
 - Driver not found → Cancel → Refund payment
- **Idempotency:** All APIs use request IDs to prevent duplicates
- **Strong Consistency:** Payment uses 2-phase commit
- **Eventual Consistency:** Search results, ratings"

Q3: How do you handle real-time location tracking at scale?

Answer: "For 100K+ active drivers:

Write Path:

1. Mobile app batches updates every 10s
2. Go service receives, validates, buffers
3. Batch write to Cassandra (100 updates/write)
4. Update Redis cache (GeoHash, TTL: 30s)
5. Publish to Kafka for analytics

Read Path:

1. Customer subscribes via WebSocket
2. Fetch from Redis (GEOPOS)

3. Calculate ETA (Google Maps Distance Matrix API)
4. Push update every 5s

Optimizations:

- Geohash precision: 6 chars (~1km)
- Only update if moved > 50m
- Protocol Buffers for compression
- Cassandra partitioned by driver_id, clustered by timestamp"

Q4: How do you design restaurant search with geo-proximity?

Answer: "We use **Elasticsearch** with geospatial indexing:

Query Logic:

1. **Geo Filter:** Within 5km delivery radius
2. **Filters:** Cuisine, rating, delivery time
3. **Scoring:**
 - Text relevance (BM25): 30%
 - Rating: 25%
 - Delivery time: 20%
 - Popularity: 15%
 - Distance: 10%
4. **Personalization:** Boost based on order history

Cache: Redis for popular queries (TTL: 5 min)"

Q5: How do you handle payment failures and refunds?

Answer: "Payment requires robust error handling:

Payment Flow:

1. **Pre-authorization:** Hold amount on card
2. **Order Processing:** Wait for restaurant confirmation
3. **Capture:** Capture payment after food prepared
4. **Settlement:** Split (platform fee, restaurant revenue)

Failure Scenarios:**Gateway Timeout:**

- Retry with exponential backoff (3 attempts)
- Try backup payment method
- If all fail, cancel order

Payment Captured but Order Failed:

- Automatic refund initiation
- Refund to original method
- If refund fails, manual intervention + support ticket

Refund Scenarios:

- Restaurant cancellation: Full refund + delivery fee
- Customer cancels before preparation: Full refund
- After pickup: No refund

Implementation:

- Idempotency keys prevent duplicate charges
- Webhook handling (async via Kafka)
- Nightly reconciliation job
- Audit log for every state change"

Q6: How do you implement surge pricing during peak hours?

Answer: "Dynamic pricing balances supply-demand:

Formula:

```
surge_multiplier = base * (1 + demand_factor + supply_factor)

demand_factor = (current_orders - avg_orders) / avg_orders
supply_factor = (required_drivers - available_drivers) / required_drivers

Capped at 3x max
```

Implementation:

1. Calculate every 5 minutes per zone
2. Gradual changes (max 0.2x increase per interval)
3. Clear notification to customers
4. A/B testing for optimization

Advanced:

- Predictive surge (ML predicts 30 min ahead)
- Loyal customers get lower surge
- Premium members get capped surge

Challenges:

- Customer backlash → Clear communication
- Driver gaming → Surge based on accepted orders, not online count"

Q7: How do you handle high-traffic events (10x spike)?

Answer: "Preparation strategy:

Before Event:

1. Pre-provision 3x capacity

2. Load testing with Gatling
3. Feature flags to disable non-critical features
4. Aggressive rate limiting

Degradation Strategy:

- Serve stale cache if DB overloaded
- Queue non-critical updates
- Disable real-time ETA, show static estimate
- Place orders in queue if overloaded, process FIFO

Monitoring:

- Real-time dashboards (order rate, latency, errors)
- Auto-scaling at 70% capacity
- Alerts for anomalies"

Q8: How do you handle the three-party coordination problem?

Answer: "Unlike two-party systems (YouTube), Zomato has Customer-Restaurant-Driver:

Challenge:

- Order can fail at any stage (payment, restaurant, driver)
- Need distributed transaction across 3 services

Solution: Saga Pattern

```
Step 1: Order Service creates order → PENDING
Step 2: Payment Service charges → PAID
Step 3: Restaurant Service confirms → CONFIRMED
Step 4: Delivery Service assigns → ASSIGNED
```

If any step fails:

- Compensating transactions rollback previous steps
- Customer gets refund
- Restaurant notified of cancellation

Implementation:

- Each service publishes events to Kafka
- State machine tracks order lifecycle
- Timeout handlers for stuck orders
- Manual intervention queue for edge cases"

SECTION 7: MONITORING & OBSERVABILITY

7.1 Key Metrics

Service Level Indicators (SLIs):

- Order placement success rate: > 99.9%
- Search latency (p99): < 500ms
- Order status update latency: < 2s
- Payment success rate: > 99.5%
- Driver assignment time: < 30s

Business Metrics:

- Orders per minute
- Average order value
- Delivery time (actual vs estimated)
- Driver utilization rate
- Restaurant acceptance rate
- Customer cancellation rate

7.2 Alerting

Critical Alerts (PagerDuty):

- Payment gateway down
- Order service latency > 5s
- Database connection pool exhausted
- Kafka consumer lag > 10K messages
- No available drivers in major cities

Warning Alerts (Slack):

- Cache hit rate < 80%
- API error rate > 1%
- Slow queries > 1s

SECTION 8: TECHNOLOGY STACK

8.1 Service Stack

- **Java/Spring Boot:** Order, Payment (transactional)
- **Go:** Location, Delivery (high throughput)
- **Python:** Search, Analytics, ML
- **Node.js:** User, Notification (I/O intensive)

8.2 Data Stores

- **PostgreSQL:** Orders, Users, Payments (ACID)
- **MongoDB:** Restaurants, Menus (flexible schema)
- **Cassandra:** Location tracking (time-series)

- **Redis:** Cache, Sessions, Geospatial
- **Elasticsearch:** Restaurant search

8.3 Message Queue

- **Apache Kafka:** Order events, Location updates, Notifications

8.4 Infrastructure

- **Cloud:** AWS/GCP
- **Container Orchestration:** Kubernetes (EKS/GKE)
- **Load Balancer:** ALB/Nginx
- **CDN:** CloudFront
- **Storage:** S3 for images
- **Monitoring:** Prometheus + Grafana, ELK Stack

APPENDIX: QUICK REFERENCE

System at a Glance

Scale:	50M DAU, 14M orders/day, 810 orders/sec peak
Microservices:	8+ services (User, Order, Restaurant, Delivery, Location, Payment, Search, Notification)
Databases:	PostgreSQL, MongoDB, Redis, Cassandra, Elasticsearch
Message Queue:	Kafka (order-events, location-updates, notifications)
Real-time:	WebSockets for live tracking
Caching:	Multi-level (CDN, Redis, App cache)

Key Trade-offs Discussed

1. **ACID vs Eventual Consistency:** Strong for orders/payments, eventual for search
2. **Microservices vs Monolith:** Microservices for independent scaling
3. **Sync vs Async:** Async for notifications, sync for order placement
4. **SQL vs NoSQL:** PostgreSQL for transactions, MongoDB for flexible schemas
5. **Push vs Pull:** Push notifications for real-time updates
6. **Batch vs Real-time:** Batch location writes, real-time reads

Must-Mention Points (Unique to Zomato)

✅ Saga pattern for distributed transactions (3-party coordination) ✅ Redis GeoRadius for driver assignment (location-based) ✅ Real-time location tracking with Cassandra + WebSocket ✅ Order state machine with cancellation policies ✅ Surge pricing algorithm (supply-demand balancing) ✅ Polyglot persistence: PostgreSQL (orders), MongoDB (menus), Cassandra (tracking)

END OF INTERVIEW GUIDE

This guide follows YouTube-style system design interview format: Requirements → Architecture → Workflows → Scaling → Trade-offs

No database schemas, no code implementation, no LLD classes - just high-level design discussions

Total: ~650 lines | Estimated PDF: 15-18 pages